



PROPUESTA DE VALOR para el Plan estratégico 2026-2027 de Banesco Seguros Venezuela

Módulo de Formas Configurables: nuevo acercamiento de Desarrollo de Aplicaciones

Es una librería de componentes FrontEnd y Backend que permite crear secuencias completas de pantallas de captura de datos para problemas de negocio diversos, a partir de archivos de configuración JSON registrados en tablas de la base de datos. La idea central es poder **construir aplicaciones nuevas** o **actualizar aplicaciones existentes** de una forma **fácil y confiable**: sin depender de red despliegues para cada cambio de pantalla o de negocio, y con un mismo modelo de configuración para todos los flujos. Esto se traduce en **reducción del tiempo de desarrollo** y en una **evolución ágil de los flujos de captura** mediante configuración en lugar de programación, lo que permite adaptar con mayor velocidad las soluciones a nuevos productos o procesos.

1

Ciclo de Desarrollo de Aplicaciones basadas en Configuración Json

El flujo para construir y desplegar una aplicación con Formas Configurables se basa en un ciclo de desarrollo orientado a configuración:

1. Definición del esquema JSON — Se diseña el archivo de configuración JSON que describe la secuencia de pantallas y el comportamiento completo de cada una, incluyendo el orden de presentación de las mismas. La estructura sigue una jerarquía: Forma Dinámica define los tipos de entidad (por ejemplo, Contratante, Asegurado, Vehículos, Pólizas, Siniestros, Ordenes Médicas, etc.); dentro de cada forma, las **filas** organizan el diseño visual; cada fila contiene **grupos de campos**, y dentro de estos se declaran los **campos** individuales. Para cada campo se especifican el tipo de presentación (Lista de Valores, Texto, Fecha, Checkboxes, Radio, etc.), Tipo de Dato, Etiquetas por idioma, Funciones de validación (Obligatorio, Formato de entrada, Validaciones personalizadas), Mecanismos de filtrado entre campos, así como Comportamientos para lógica

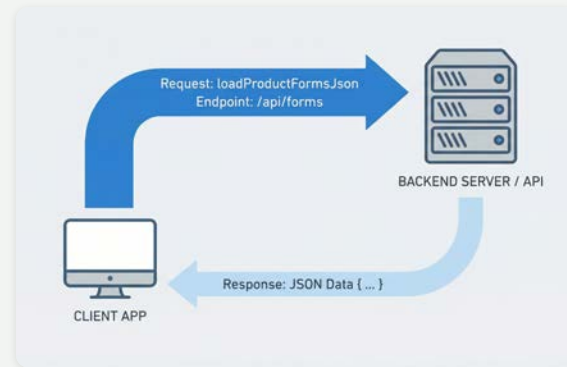
condicional de presentación de los campos (ocultar o mostrar campos según el valor ingresado en otros, etc.). La configuración permite además un **control completo de la interfaz**: distribución de componentes en pantalla, carrusel de navegación, orden de presentación de las pantallas, botones de navegación y demás elementos de la experiencia de usuario. Todo ello en un único archivo JSON, sin escribir código de UI por pantalla. Además, siempre es posible incluir, de forma dinámica, componentes personalizados que sean propios de la aplicación que se está construyendo, sin necesidad de incorporarlos a la librería.

```
"attributes": {
  "Sintoma": {
    "desc": "Sintoma",
    "order": 1,
    "typevisual": "selectsearch",
    "typedata": "String",
    "use": "input",
    "visible": true,
    "editable": true,
    "grouping": "multiple",
    "style": "width: 340px;",
    "labels": {
      "es": { "label": "Síntoma que presenta", "language": "es" }
    },
    "helps": {
      "es": { "label": "Indique el Síntoma que presenta", "language": "es" }
    },
    "validators": [
      {
        "type": "validate",
        "customClass": "ibforms_required",
        "customMethod": "return this.validators.ibforms_required(value, this.attrName, this.formsInputs, 'Debe especificar un sintoma válido');",
        "messages": {},
        "varValidators": []
      }
    ],
    "filters": [
      { "filterAttribute": "Sintoma", "filterbyInput": false, "filterColumn": "", "relation": "LIKE", "obligatory": false },
    ]
  }
}
```

2. Registro en Tablas — La configuración se persiste en la base de datos en tablas, de modo que la definición vive en datos y no en código. Las actualizaciones que se necesiten pueden hacerse directamente sobre las tablas correspondientes —por ejemplo, cambiar una etiqueta, un texto de ayuda o un estilo de presentación— sin tocar el código de la aplicación. Otras ventajas de persistir la configuración en tablas: **evitar redespliegues** para cada cambio de negocio o de textos; **gestionar entornos** (desarrollo, pruebas, producción) con configuraciones distintas según el ambiente; **centralizar** la definición de formas



en un único repositorio consultable y auditable; y **permitir correcciones o rollbacks** mediante actualizaciones o restauración de datos, sin pasar por un nuevo despliegue.



3. Carga dinámica por el backend — La API de ibForms está desarrollada en Spring Boot y es la responsable de leer la configuración desde las tablas y exponerla en formato JSON para que el frontend construido en React Hook Form pueda desplegar los componentes. Para esto se dispone de diferentes endpoints que permiten entregar esa configuración a los clientes en formato JSON. Además, la API de ibForms centraliza el despliegue de los distintos proyectos: todos los módulos que utilizan Formas Configurables basadas en configuración JSON consumen esta misma API, de modo que la lógica de carga y entrega de configuración queda unificada.

4. Renderizado por DynamicForm — El componente **DynamicForm** (ibForms) está construido con **React Hook Form**, lo que permite gestionar el estado de los formularios, la validación y el envío de datos de forma eficiente y alineada con la configuración JSON. A partir del JSON recibido del backend, DynamicForm interpreta la estructura (campos, tipos, validaciones, listas de valores, manejo de eventos, búsqueda de datos, etc.) y despliega las Formas de manera dinámica: componentes de la librería ibForms generan la UI sin código específico por pantalla, reutilizando la misma lógica de renderizado para cualquier flujo definido en configuración.



5. Componentes específicos extensibles — Sobre la base del mismo esquema de configuración se pueden incorporar componentes especializados (pantallas de **Resumen**, tablas maestro-detalle, wizards multi-paso) que mantienen la coherencia y la reutilización. La librería ibForms permite extender el catálogo de componentes para dominios concretos sin romper el modelo de configuración.

6. Consultas basadas en fuentes de datos — Posibilidad de diseñar consultas de datos que invocan **funciones de base de datos** para obtener el conjunto de datos a mostrar. Las consultas se construyen también a partir de una **configuración JSON** donde se especifican: filtros de la consulta, tabla de resultados, API de búsqueda de datos, paginación, etc.

Las fuentes de datos se configuran de forma declarativa: se asocian a tipos de entidad o pantallas concretas y el backend ejecuta las funciones configuradas para alimentar listas desplegables, datos precargados o pantallas de detalle. Esto permite reutilizar lógica de negocio ya existente en la BD (catálogos, parámetros, reglas) sin duplicar código en la aplicación.



7. Mantenimiento sencillo — Una vez desplegadas las Formas, el mantenimiento se realiza modificando únicamente, en la mayoría de los casos, los valores de configuración en las tablas, sin cambios en el código de la aplicación. Cambios de etiquetas, textos de ayuda, orden de campos, validaciones o nuevos pasos en un flujo se resuelven actualizando la configuración y, si aplica, los datos en BD, reduciendo el tiempo de entrega y el riesgo asociado a redesplices.

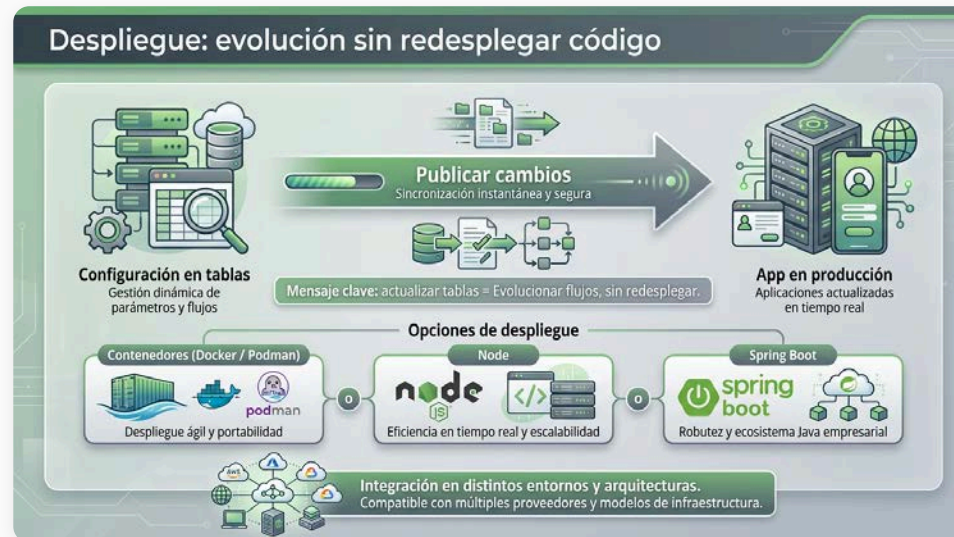
Mantenimiento sencillo: Actualizar configuración en tablas

Formas Configurables



Es posible ajustar tipos de presentación, listas de valores, comportamientos condicionales y reglas de negocio declaradas en la configuración sin tocar el frontend ni el backend; en muchos casos, perfiles funcionales o equipos de producto pueden asumir estas actualizaciones con un conocimiento mínimo técnico. La trazabilidad de los cambios queda en la base de datos, lo que facilita auditorías y, en su caso, rollbacks controlados.

8. Despliegue — Las pantallas basadas en configuración JSON permiten construir pequeñas aplicaciones React cuyo comportamiento principal está definido en esos archivos de proyecto JSON. Una vez registrada la configuración, basta con publicar los cambios en las tablas para evolucionar flujos sin redespregar. Estas aplicaciones pueden desplegarse en contenedores (Docker o Podman), ejecutándose sobre Node o como aplicaciones Spring Boot, lo que facilita su integración en distintos entornos y arquitecturas.



En resumen: *configuración JSON* → *persistencia en Tablas* → *DynamicForm las despliega como Formas Configurables*. El ciclo es ágil y centrado en datos.

2 Construcción de Aplicaciones React basadas en módulo de configuración de Formas y Consultas ibForms

El siguiente esquema resume la idea: módulos por dominio, reutilización con ibForms y despliegue e integración.

Módulos y sistema. Los sistemas complejos se organizan en varios módulos (cotizaciones, órdenes médicas, siniestros, solicitudes, expedientes, seguimientos, etc.), donde se muestran pantallas que tienen elementos de presentación y comportamiento muy similares: consultas, captura por pasos, validaciones, comportamiento condicional de campos y listas de valores.



Comúnmente estos módulos pueden construirse usando herramientas de IA para generación de código, lo que permite obtener gran cantidad de componentes que despliegan la funcionalidad requerida. Sin embargo, este enfoque, aunque funcional, presenta carencias importantes:

- **Poca reutilización entre módulos:** los componentes comunes (formularios, tablas, validaciones) se duplican o se generan de forma distinta en cada módulo, sin un núcleo compartido.
- **Problemas repetidos:** los mismos retos (navegación por pasos, listas de valores, reglas condicionales) pueden estar resueltos una y otra vez en cada módulo, con el consiguiente riesgo de incoherencias y bugs.
- **Mantenimiento más complejo:** el gran volumen de componentes generados hace que evolucionar o corregir el sistema requiera tocar muchos archivos y módulos, lo que encarece el mantenimiento y aumenta el riesgo de regresiones.

Frente a esto, el modelo de Formas Configurables centraliza la lógica de presentación y comportamiento en una librería (ibForms) y en configuración (JSON en tablas), reduciendo la cantidad de componentes a mantener y favoreciendo la reutilización en todos los módulos.

Librería ibForms y reutilización. Construir cada módulo de forma independiente y homogénea: formularios, tablas, navegación, validación y enlace a datos están encapsulados en la librería.

La generación y el mantenimiento de los proyectos de configuración JSON pueden apoyarse en herramientas de IA, alimentadas con el modelo de configuración existente. Al tener el esquema (formas dinámicas, filas, grupos, campos, validaciones) como referencia, las herramientas pueden proponer o completar configuraciones a partir de descripciones en lenguaje natural o de ejemplos ya desplegados, reduciendo el tiempo de creación y la probabilidad de errores de transcripción. El resultado se mantiene alineado con el modelo que ibForms espera, de modo que la evolución de los JSON sigue siendo ágil y coherente con la librería.



¿Cómo construir la configuración? Para crear los proyectos de configuración JSON que alimentan el módulo de Formas Configurables **no se requiere una herramienta de**

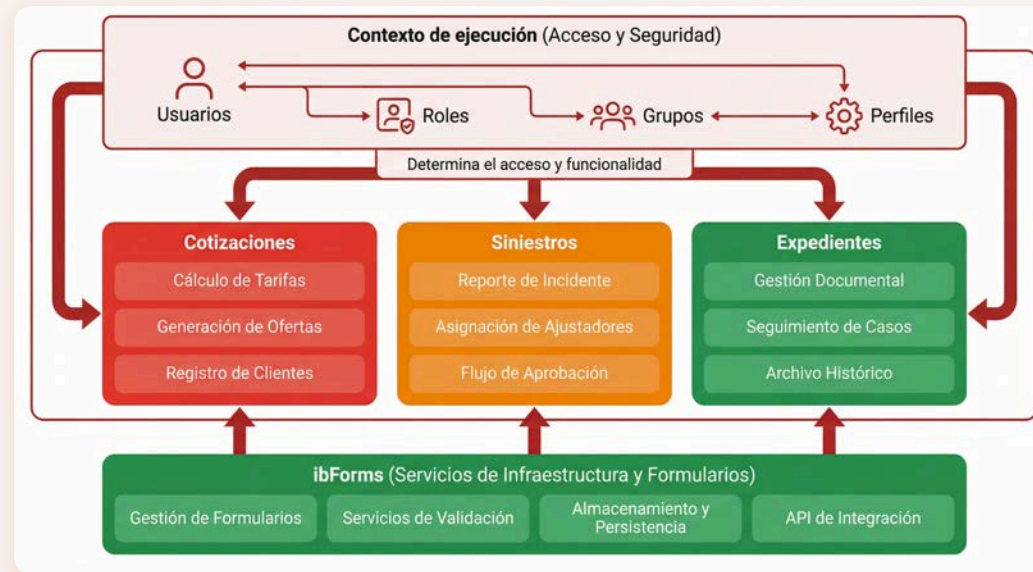
configuración propietaria: basta con definir los archivos JSON. Las opciones son flexibles y se adaptan al equipo y al contexto:

- **Creación manual** — Edición directa de los JSON siguiendo el esquema del modelo (formas dinámicas, filas, grupos, campos, validaciones, comportamientos). Ideal para ajustes finos o cuando ya se conoce bien la estructura.
- **Agentes de IA a partir de texto** — Descripciones en lenguaje natural de las pantallas o flujos que se desean (por ejemplo: «formulario de contratante con nombre, RFC, domicilio y teléfono») pueden alimentar agentes de IA que generen o completar la configuración JSON, reduciendo tiempo y errores de transcripción.
- **Agentes de IA a partir de imágenes** — Capturas de pantalla de aplicaciones existentes que se quieren migrar, wireframes o prototipos elaborados en HTML u otras herramientas sirven como entrada para que la IA proponga o genere la configuración JSON correspondiente, acelerando la modernización y la réplica de pantallas sin reescribir código.

En todos los casos, el resultado es el mismo: archivos JSON que se registran en las tablas y que el módulo de Formas Configurables interpreta sin necesidad de herramientas adicionales.



Despliegue e integración. Las aplicaciones se construyen sobre **React** (frontend) y **Spring Boot** (backend y API de ibForms), lo que facilita su despliegue y ejecución en entornos estándar. Los módulos se integran como partes del sistema global y pueden desplegarse de forma autónoma en contenedores (p. ej. Docker), favoreciendo la estabilidad y la escalabilidad. Cada módulo consume las APIs de ibForms y puede exponer APIs propias que personalicen reglas de dominio del negocio.



La combinación de **Spring Boot** y **React** es un estándar de la industria para aplicaciones modernas y robustas. La arquitectura desacoplada permite que backend y frontend evolucionen de forma independiente, comunicándose mediante una API REST.

Sus principales fortalezas incluyen:

- **Seguridad empresarial:** Spring Security, JWT, OAuth2, control por roles.
- **Rendimiento:** Virtual DOM en React, procesamiento eficiente en Spring Boot.
- **Productividad:** separación de responsabilidades, componentes reutilizables, ecosistema maduro.
- **Preparación para la nube:** microservicios, contenedores con Docker, despliegue en AWS o Azure.

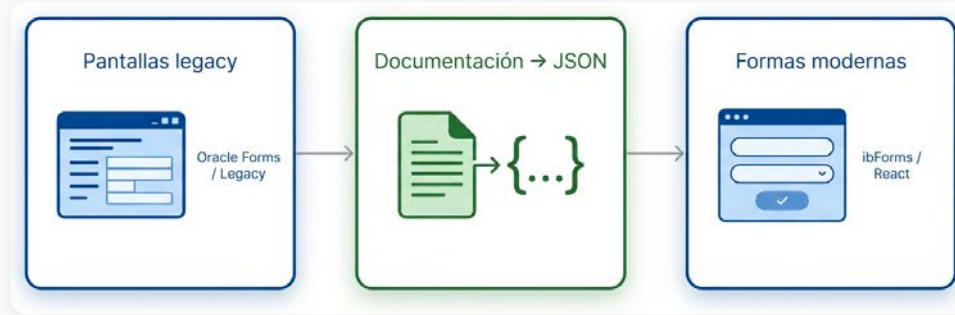
Beneficios: Comparativa de desarrollo tradicional vs Formas Configurables basadas en Json

Ventajas del modelo basado en configuración frente a desarrollar cada pantalla como un caso particular:

Aspecto	Desarrollo tradicional	Formas Configurables
Nueva pantalla o flujo	Código nuevo por pantalla, pruebas y despliegue	Definición JSON + registro en tablas; sin redespigue
Cambio de textos o validaciones	Modificar código, redespigar	Actualizar configuración en BD
Reutilización entre módulos	Copiar/pegar o librerías ad hoc	Misma API y DynamicForm para todos los proyectos
Mantenimiento y auditoría	Repositorio y versionado de código	Configuración en datos, consultable y auditable

Beneficio principal: Reducción del tiempo de desarrollo y evolución de flujos de captura mediante configuración en lugar de programación. Mayor velocidad para adaptar soluciones a nuevos productos o procesos.

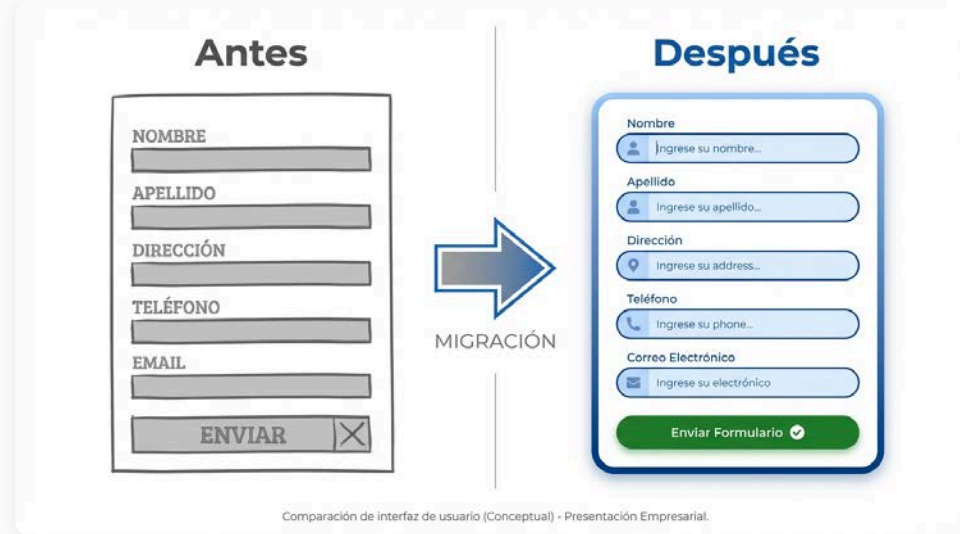
El modelo de Formas Configurables no se limita a aplicaciones nuevas: es una vía práctica para **migrar con rapidez** sistemas construidos en frameworks antiguos, como **Oracle Forms** u otros entornos basados en formularios.



El flujo típico de migración puede ser el siguiente:

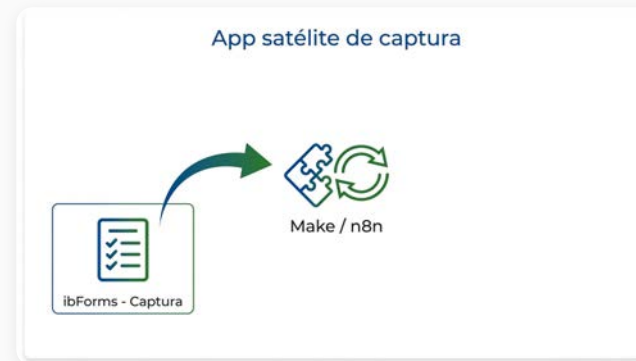
- **Captura y documentación.** Se toman capturas de las pantallas existentes y se documentan los elementos: campos (tipo, etiqueta, obligatoriedad), listas de valores asociadas (LOV), validaciones, formato de entrada/salida y dependencias entre campos (comportamiento condicional, habilitar/ocultar). Esta documentación puede hacerse de forma manual o apoyándose en agentes de IA que, a partir de las imágenes y de una plantilla, propongan la estructura.
- **Generación de la configuración JSON.** A partir de esa documentación se genera el archivo (o los archivos) de configuración JSON que describe las formas dinámicas, filas, grupos y campos en el esquema que consume el módulo. De este modo, cada pantalla legacy se traduce en configuración, no en código nuevo.
- **Despliegue en el nuevo modelo.** La configuración se registra en las tablas y se sirve vía API; el frontend basado en ibForms despliega las pantallas. El resultado son las mismas funcionalidades (captura, validaciones, listas de valores) en una aplicación moderna, con menor esfuerzo que una reescritura pantalla a pantalla.

Este enfoque acerca el concepto de la librería a **casos reales de modernización**: reducir tiempo y riesgo en proyectos de migración, reutilizando un mismo motor de formularios y una misma forma de definir el comportamiento por configuración.



4 Integración con plataformas de flujos de trabajo (Make, n8n, etc.)

Las plataformas de integración y automatización de flujos —como **Make** o **n8n**— orquestan procesos entre sistemas (notificaciones, escritura en BD, envío a CRMs, etc.), pero suelen requerir **aplicaciones modulares** que capturen los datos que luego alimentan esos flujos. Es decir: alguien debe introducir la información en alguna interfaz; esa interfaz envía un payload (por ejemplo JSON) a un webhook o API, y la plataforma dispara el workflow a partir de ese dato.



Las aplicaciones construidas con **ibForms** encajan de forma natural en ese papel:

- **Captura configurable.** Los formularios se definen por JSON (campos, validaciones, listas de valores); no hace falta desarrollar una app distinta por cada flujo de captura.
- **Salida estándar.** Al enviar el formulario, el backend (o el frontend) dispone de los datos estructurados. Basta con enviar ese payload a la **URL del webhook** que proporciona Make o n8n, o a una API que esas plataformas consuman.

- **Integración rápida.** La conexión es **HTTP + JSON**: no se requiere un conector propietario. El flujo en Make o n8n se activa con «Webhook» o «HTTP Request» y utiliza los campos del JSON en los pasos siguientes (notificaciones, bases de datos, otros sistemas).

En resumen: las aplicaciones ibForms pueden actuar como **app satélite de captura** para Make, n8n o herramientas similares, permitiendo integrar nuevos flujos de trabajo de forma rápida y sin desarrollar integraciones ad hoc por cada formulario.



5 Resumen

El **Módulo de Formas Configurables** es una librería FrontEnd y Backend que permite construir y evolucionar aplicaciones de captura y flujos de negocio a partir de **configuración JSON registrada en tablas**, reduciendo tiempo de desarrollo y dependencia de red despliegues. A partir de lo expuesto en los puntos anteriores, se recogen los aspectos clave:

- **Ciclo de desarrollo:** definición del esquema JSON (formas dinámicas, filas, grupos, campos, validaciones, comportamientos condicionales y control completo de la interfaz) → registro en tablas → API de ibForms (Spring Boot) expone la configuración → **DynamicForm** (React Hook Form) despliega las Formas de manera dinámica, sin código por pantalla. Incluye consultas basadas en fuentes de datos configurables (funciones de BD, filtros, paginación), mantenimiento mediante actualización de configuración en tablas (con trazabilidad y rollback) y despliegue en contenedores (Docker, Podman) sobre Node o Spring Boot, evolucionando flujos sin red desplegar.
- **Construcción con ibForms:** los sistemas se organizan en módulos por dominio (cotizaciones, órdenes médicas, siniestros, etc.); la librería centraliza formularios, tablas, navegación, validación y enlace a datos, evitando duplicación y favoreciendo reutilización. La configuración JSON puede crearse de forma manual o con apoyo de IA (desde texto o desde imágenes). Las aplicaciones se

construyen sobre React y Spring Boot, con despliegue e integración estándar (seguridad, rendimiento, preparación para nube). Frente al desarrollo tradicional, el modelo basado en configuración permite nuevas pantallas o cambios de textos y validaciones sin redespiegue, con misma API y DynamicForm para todos los proyectos y configuración auditable en datos.

- **Migración desde legacy:** el modelo permite migrar con rapidez sistemas antiguos (p. ej. Oracle Forms): documentación de pantallas existentes (con apoyo de IA si se desea) → generación de la configuración JSON → registro y despliegue en ibForms. El resultado son las mismas funcionalidades en una aplicación moderna, con menor esfuerzo que una reescritura pantalla a pantalla.
- **Integración con plataformas de flujos:** las aplicaciones ibForms pueden actuar como **app satélite de captura** para Make, n8n o similares: formularios definidos por JSON, salida estándar en payload a webhook o API, e integración vía HTTP + JSON sin conector propietario, permitiendo orquestar procesos (notificaciones, BD, CRMs) a partir de los datos capturados.

En conjunto, el módulo centraliza la lógica de presentación y comportamiento en configuración y en una librería reutilizable (ibForms), favoreciendo la coherencia entre módulos, la evolución ágil de los flujos y su adaptación a nuevos productos o procesos sin multiplicar el esfuerzo de desarrollo.

Plan Estratégico 2026-2027 — Estimación y abordaje con Formas Configurables

A partir de las iniciativas estratégicas planteadas (plataforma omnicanal, workflows de servicios médicos y automóvil, multicotizadores y modernización de suscripción), se presenta una visión de **estimación de esfuerzo** y de **cómo el Módulo de Formas Configurables** permite abordarlas de forma eficiente, reutilizando un mismo motor de formularios y flujos por configuración.

1

Servicios a potenciar.

- **Servicios Médicos.** Se busca contar con **aplicaciones modulares** que permitan capturar el registro de solicitudes de servicios médicos, cada una orientada a un tipo de trámite: **claves de emergencia, carta aval, reembolso, APS, medicamentos e indemnizaciones.**
- **Servicios de Automóvil.** Se busca contar con **aplicaciones modulares** que permitan capturar el registro de solicitudes de servicios de automóvil, cada una orientada a un tipo de trámite: **pérdidas parciales, pérdidas totales, responsabilidad civil e indemnizatorio.**
- **Multicotizadores : Urgencias y Emergencias, Combinado residencial, Combinado industria y comercio paquetizado, Productos de personas (AP, Funerario, Cáncer, IDH y Vida).** Experiencia omnicanal, cotización en tiempo real, comparación paramétrica, suscripción y emisión, pago integrado, tracking. *Abordaje:* Los pasos de captura se modelan como **Formas Configurables** por producto o línea; el motor de cotización y las APIs se integran vía backend. **Estimación:** Configuración base por línea (3–5 semanas) más integraciones (motor de cotización, pasarelas, tracking), reutilizables entre líneas.
- **Workflow de Suscripción Etapa 1 y 2.** Modernización y configuración de productos (AP Escolar, AP Individual, Funerario Colectivo, RC Patronal, Fraude; y en Etapa 2: AP Colectivos, Combinado industria y comercio, RC Empresarial, TRI), multimonedas (VES/USD), automatización de reglas (primas, inclusión/exclusión, renovaciones, reservas), evolución de tarificación, módulos de autogestión. *Abordaje:* Cada producto se traduce en **configuración de formas** y reglas en backend; la capa de captura queda unificada en ibForms. **Estimación:** 2–4 semanas por producto modernizado (configuración e integración).
- **Servicios patrimoniales y ramos de personas;** multicotizadores para combinado industria/comercio y productos de personas (AP, Funerario, Cáncer, IDH, Vida). El mismo modelo aplica: **nuevos flujos = nueva configuración JSON**; las estimaciones por línea se benefician de la reutilización de configuraciones y patrones ya desplegados.

2

Características a considerar.

Estas aplicaciones deben integrarse en un **workflow único** que permita hacer el seguimiento de las solicitudes de punta a punta, con las siguientes características:

- **Roles de usuario:** empleado, proveedor, asegurado e intermediario, cada uno con visibilidad y acciones acordes a su rol dentro del flujo del proceso. A lo largo del ciclo de vida del portal Banesco

Seguros se ha consolidado una estructura de usuarios basada en **Roles, Perfiles y Privilegios**, que permite configurar las necesidades de estas funcionalidades; apoyarse en estas estructuras aportaría un valor agregado muy importante a los nuevos módulos a desarrollar.

- **Dashboard de seguimiento de gestión:** panel centralizado donde los distintos usuarios (empleados, supervisores, gestores) pueden consultar el estado de las solicitudes bajo su ámbito de responsabilidad. Las vistas se filtran por **rol, estado de la solicitud**, tipo de trámite (salud, automóvil, etc.) y, si aplica, por asignación o período, de modo que cada perfil acceda solo a la información relevante. El dashboard facilita el seguimiento de las solicitudes de punta a punta, la priorización de tareas y la identificación de cuellos de botella en el flujo de los procesos.
- **Indicadores de gestión:** medición de tiempos, volúmenes y estados del proceso, alimentados por los datos que las aplicaciones capturan o generan a lo largo de los flujos en los que interviene el usuario. El objetivo es disponer de representaciones visuales que permitan el seguimiento y el monitoreo de estos indicadores.

3 Sin comenzar desde cero, hagamos uso de lo aprendido:

Dentro del Portal Banesco Seguros Online se cuentan con funcionalidades similares, por lo cual se puede hacer la migración a este nuevo modelo de Formas Configurables, reutilizando tanto los conceptos funcionales ya existentes, como componentes de código backend que pueden convertirse rápidamente en APIs.

Esta base existente aporta ventajas claras: **simplificación de los tiempos de desarrollo**, al evitar partir de cero y poder reutilizar flujos, reglas y pantallas ya validados por el negocio; **reducción de la curva de aprendizaje**, porque equipos funcionales y técnicos ya conocen los conceptos (solicitudes, expedientes, roles, tipos de trámite), y el esfuerzo se concentra en adoptar el modelo de configuración en lugar de redescubrir el dominio; y **menor riesgo y mayor coherencia**, al mantener la misma terminología y lógica que el negocio usa hoy y permitir una migración gradual por área o por flujo. Aprovechar lo aprendido en el portal actual hace que la transición hacia Formas Configurables sea más ágil y predecible.

Funcionalidades existentes en el portal actual:

Área	Funcionalidades
Gestión de Usuarios	• Creación y Administración de Usuarios • Configuración y Gestión de Roles y Perfiles de Usuarios • Configuración y asignación de opciones de Usuarios • Monitoreo de Transacciones
Servicios Médicos	• Solicitud de Clave de Emergencia • Solicitud de Carta Aval • Solicitud de Orden Médica • Reembolsos de Salud • Notificaciones de Siniestros de Vida y Funerario
Servicios de Automóvil	• Declaración de Siniestros de Automóvil: Parcial y Total
Cotización y Suscripción	• Funerario • Accidentes Personales • Cáncer • Vida • Renovación de Certificados de Salida de Grupo BOF
Gestión de Documentos	• Carga de Documentos de Expedientes • Sincronización de Gestor Documental • Búsqueda de Expedientes existentes • Notificaciones de documentos faltantes
Creación y Seguimiento de Requerimientos	• Modificaciones de Pólizas • Renovaciones • Anulaciones

Área	Funcionalidades
Servicios de Pagos	• Débito en Cuenta • Tarjeta de Crédito • Notificación de Transferencias

4 Abordaje con Formas Configurables.

Para ambas iniciativas (Servicios Médicos y Servicios de Automóvil), cada tipo de solicitud o variante se modela como una secuencia de pantallas representadas por **formas dinámicas**, con su propio proyecto de configuración JSON (pasos, campos, validaciones, listas de valores y reglas de visibilidad por rol). Las aplicaciones modulares son instancias del mismo frontend (librería ibForms) que consumen configuraciones distintas; la reutilización del motor reduce fuertemente el esfuerzo frente a desarrollar cada pantalla como un caso particular. El workflow de seguimiento y los indicadores se alimentan de los datos registrados y pueden apoyarse en consultas configurables y en integración con el backend.

Estimación — elementos típicos del desarrollo de un módulo con Formas Dinámicas:

Elemento del desarrollo	Tiempo estimado
Diseño funcional y definición del esquema JSON (formas, filas, grupos, campos, validaciones)	1-2 semanas
Registro de la configuración en tablas	0,5-1 semana
Configuración de formas por flujo o por producto (variantes, pasos, listas de valores)	1-2 semanas
Construcción e Integración con APIs existentes y reglas de negocio (backend)	2-3 semanas

Elemento del desarrollo	Tiempo estimado
Workflow de seguimiento, dashboards e indicadores (si aplica)	2-3 semanas
Pruebas y puesta en producción	1-2 semanas

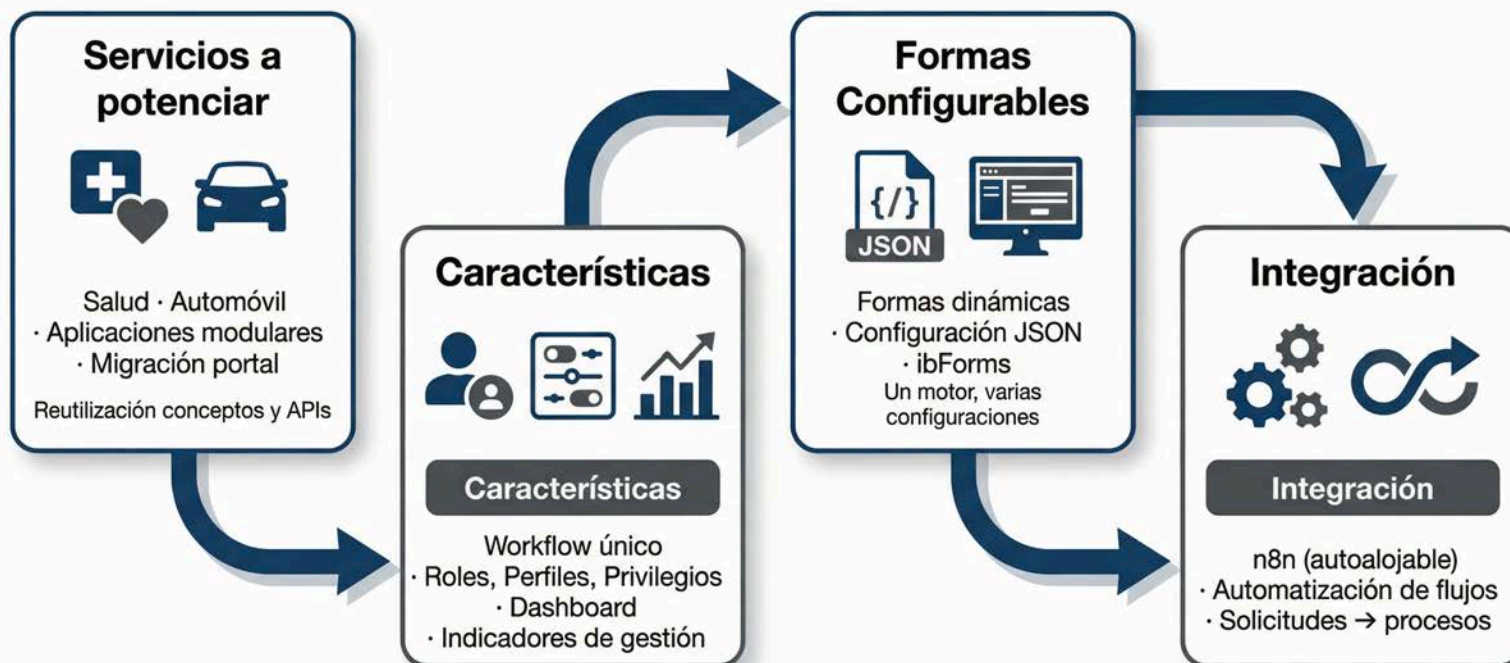
Los rangos son orientativos; el esfuerzo total por iniciativa dependerá del número de flujos o productos y del grado de reutilización.

Al centralizar la construcción de pantallas y flujos en **configuración** y en una **librería común** (ibForms), el esfuerzo se desplaza de "desarrollar N pantallas por producto" a "diseñar y mantener N configuraciones", con tiempos de entrega y riesgo menores. Los presupuestos pueden estructurarse por fases (diseño funcional, configuración por flujo/producto, integración backend, pruebas y puesta en producción), permitiendo priorizar iniciativas y escalar el alcance sin multiplicar el costo por pantalla.

5 Integración con plataformas de automatización de flujos de trabajo.

Los módulos de servicios desarrollados a partir del **abordaje con Formas Configurables** (captura de solicitudes de salud y automóvil, workflows de seguimiento) pueden integrarse con plataformas de automatización como **n8n**, recomendada por ser una herramienta que se puede autoalojar, de modo que los flujos de negocio no queden limitados a la aplicación web: las solicitudes registradas pueden disparar procesos posteriores (notificaciones, validaciones con terceros, actualización de sistemas), y a la inversa, flujos diseñados en esas plataformas pueden invocar los módulos de captura como pasos del proceso. Así se automatizan flujos de trabajo asociados al negocio y se aprovecha la misma base de **formas dinámicas** y APIs ya expuestas por el módulo.

Iniciativas 2026-2027



Resumen visual de Iniciativas 2026-2027: servicios a potenciar (Salud y Automóvil), características (roles, dashboard, indicadores), abordaje con Formas Configurables e integración con n8n.